



UNIVERSIDAD
NACIONAL DE SAN MARTÍN

GUÍA DE EJERCICIOS INTEGRADORES

Preparación para el Parcial

1 Ejercicios Integradores

1.1 Sistema de Biblioteca Digital

Crea una clase `Libro` con atributos título, autor y año. Implementa una clase hija `LibroDigital` que agregue el atributo URL de descarga.

- Usa un decorador para registrar cada vez que se descarga un libro (mostrar título y fecha).
- Genera un diccionario por comprensión que contenga `título: año`.
- Guarda y lee el catálogo en formato JSON usando manejo de archivos y excepciones.

1.2 Gestión de Estudiantes y Notas

Crea una clase `Estudiante` con un diccionario de materias y notas.

- Usa una función lambda con `map` para calcular el doble de todas las notas.
- Crea una comprensión de listas que filtre las notas mayores a 7.
- Implementa un método que guarde las notas en un CSV y otro que las lea, manejando excepciones si el archivo no existe.

1.3 Sistema de Ventas Online

Diseña una clase `Producto` con atributos nombre y precio. Implementa subclases `Alimento` y `Electrónico` con su propio cálculo de impuesto.

- Usa una comprensión de conjuntos para almacenar los nombres únicos de los productos vendidos.
- Implementa un generador que devuelva los productos con precio mayor a 10.000.
- Guarda un registro de ventas en un archivo de texto con fecha y hora.

1.4 Aplicación de Tareas

Crea una clase `ListaDeTareas` con una lista de tareas.

- Usa un decorador que limite a un máximo de 5 tareas pendientes.
- Genera un conjunto con las primeras letras de cada tarea usando comprensión de conjuntos.
- Implementa manejo de excepciones al intentar eliminar una tarea que no existe.

1.5 Simulación de Zoológico

Crea una clase abstracta `Animal` con el método `hacerSonido()`. Implementa `Perro`, `Gato` y `Pájaro`.

- Usa polimorfismo llamando a `hacerSonido()` en una lista de animales.
- Implementa un generador que devuelva los nombres de los animales de a uno.
- Guarda en un archivo binario un listado con los sonidos y luego léelo.

1.6 Sistema de Reservas de Vuelos

Crea una clase `Vuelo` con número, origen y destino.

- Implementa un decorador que valide si el destino no está vacío antes de guardar la reserva.
- Usa una comprensión de diccionario para mapear `número de vuelo:` `destino`.
- Maneja la excepción si se intenta registrar un vuelo con un número repetido en un archivo CSV.

1.7 Análisis de Datos Meteorológicos

Genera un archivo CSV `mediciones.csv` con columnas tiempo y temperatura.

- Usa un generador que devuelva solo las temperaturas mayores a 25.
- Calcula con una lambda el promedio de las temperaturas.
- Maneja excepciones para división por cero si el archivo está vacío.